

ISM2411 — Lab Week 14

Clean a Messy Sales CSV — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 14 · Unit 4 · Data Analysis with pandas

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Segments	3
6 Segment-by-Segment Walkthrough	4
6.1 Intro (0:00–0:04)	4
6.2 Exercise 1 — Inspect (0:04–0:12, 8 min)	4
6.3 Exercise 2 — Standardize Column Names (0:12–0:18, 6 min)	7
6.4 Exercise 3 — Handle Missing Values (0:18–0:28, 10 min)	9
6.5 Exercise 4 — Fix Types (0:28–0:38, 10 min)	12
6.6 Exercise 5 — Check Duplicates (0:38–0:44, 6 min)	14
6.7 Exercise 6 — Describe (0:44–0:53, 9 min)	16
6.8 Exercise 7 — Save (0:53–1:01, 8 min)	18
7 Wrap-Up (last ~5 minutes)	20
8 Appendix A — Full Answer Key (clean.py)	20
9 Appendix B — Reproducible Messy Dataset (for instructor testing)	22
10 Appendix C — Extra Practice (only if the class finishes early)	23

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 14 Lab — Clean a Messy Sales CSV
Unit	Unit 4 · Data Analysis with pandas
Class length	75 minutes
Format	Live code-along
Prerequisites	Module 13: DataFrames, boolean filtering, <code>.dtypes</code>
Student-facing lab page	markumreed.github.io/ism2411 — week14_lab
Exercises covered	Exercises 1–7 (required)
Submission	<code>clean.py</code> (or <code>clean.ipynb</code>) + <code>clean_sales.csv</code> via GitHub (module14/ folder), URL to Canvas

Module 13 quietly assumed the data was already clean. This module confronts the fact that real data almost never starts that way — and the lab page’s own framing deserves repeating verbatim: “every decision you make here will feed directly into Module 15’s analysis and the capstone.” Two things make this lab different from every prior one: first, several exercises have **no single correct answer** — dropping vs. filling a missing value is a judgment call, and this lab explicitly grades the *reasoning* in students’ comments, not just working code. Second, real messy data teaches lessons no clean synthetic dataset can — protect real time for actually looking at `.info()`’s output together (Exercise 1) and for the genuinely surprising type-loss-on-reload gotcha in Exercise 7.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Use `.info()` and `.isnull().sum()` to diagnose a dataset’s specific quality problems before touching any data.
2. Decide, with articulated reasoning, when to drop rows with missing data versus fill them with a computed value.
3. Convert columns to correct types with `pd.to_datetime(errors="coerce")` and `.astype()`, understanding what `errors="coerce"` actually does to unparseable values.
4. Detect and remove duplicate rows with `.duplicated()` and `.drop_duplicates()`.
5. Read `.describe()`’s output critically, identifying statistics that suggest a real business explanation is needed, not just accepting numbers at face value.
6. Explain why saving and reloading a CSV does not always preserve every column’s dtype, and what to do about it.

3 Before Class — Setup Checklist

- ☐ Obtain or generate `data/messy_sales.csv` before class — Appendix B documents a verified, reproducible generator producing a dataset with genuine, realistic messiness: mixed-case/spaced column names, "\$"-prefixed price strings, missing values in both optional and essential columns, one malformed date string, two duplicate rows, one extreme price outlier, and one negative price (a plausible data-entry error). **Use your course's real provided dataset with students** — the numbers in this guide are verified against the synthetic version for your own preparation.
- ☐ Rehearse Exercise 7's save-then-reload sequence yourself before class — the dtype-loss-on-reload behavior is genuinely surprising even to instructors seeing it for the first time, and it's worth having your own reaction rehearsed rather than looking caught off guard.
- ☐ Decide in advance which specific missing-value columns you'll fill vs. drop for your actual dataset, and be ready to explain your own reasoning as a model answer, since Exercise 3 is explicitly a judgment call this lab wants students to articulate, not a fixed formula to apply mechanically.

4 Materials Needed

- Instructor laptop + terminal + editor, Python 3.10+, pandas installed
- Students: `data/messy_sales.csv`, same GitHub repo with a new `module14/` folder

5 Segments

The required exercises fill most of the available time.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Module 13’s dataset loaded cleanly, with no missing values, no type problems, no duplicates — that was not realistic, and I told you so at the time. Today’s dataset is deliberately messy, the way real data actually is. And I want to flag something different about this lab before we start: several of today’s decisions don’t have one single correct answer. Drop a row, or fill it with an estimate? Both can be defensible — what matters is that you can explain why you chose what you chose. Your comments today are graded as seriously as your code.”

Do: Open `clean.py`, type the header:

```
# clean.py
# ISM2411 Module 14 Lab – Clean a Messy Sales CSV
```

6.2 Exercise 1 — Inspect (0:04–0:12, 8 min)

Teaching goal: `.info()` and `.isnull().sum()` as the mandatory first step on any new, unfamiliar dataset — and writing down *specific*, evidence-based observations rather than a vague “the data looks messy.”

Say to the class:

“Before changing anything, diagnose. Two commands, and I want you to read their output like a doctor reading test results — specific numbers, not vague impressions.”

Live-code this:

```
# --- Exercise 1 ---
import pandas as pd

df = pd.read_csv('data/messy_sales.csv')
print(df.info())
print(df.isnull().sum())

# Data quality issues observed:
# 1. [fill in after reading output below]
# 2. ...
# 3. ...
```

Line-by-line explanation:

- `df.info()` — prints, for every column: its name, how many **non-null** values it has (out of the total row count), and its dtype. This single call answers two questions at once — “does this column have missing data” (compare the non-null count to the total row count) and “is this column the type I’d expect” (check the dtype column) — say explicitly that reading `.info()`’s non-null counts against the total row count, column by column, is the actual skill, not just running the command.
- `df.isnull().sum()` — a second, complementary view: `.isnull()` produces a same-shaped grid of True/False (a value is missing or it isn’t), and `.sum()` (True counts as 1, False as 0) totals that up **per column**, giving a direct missing-value count rather than requiring the subtraction `.info()`’s non-null counts implicitly ask for.

Run it against your dataset. Expected output shape (verified against the synthetic sample in Appendix B — actual counts will differ with real course data, but the *pattern* — mismatched dtypes, non-null counts below the total, a nonzero `.isnull().sum()` on several columns — should look structurally similar):

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 42 entries, 0 to 41
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	Order ID	41 non-null	object
1	Order Date	41 non-null	object
2	Product Category	42 non-null	object
3	Region	42 non-null	object
4	Unit Price	40 non-null	object
5	Quantity Sold	42 non-null	int64
6	Customer ID	42 non-null	object

Order ID	1
Order Date	1
Product Category	0
Region	0
Unit Price	2
Quantity Sold	0
Customer ID	0

Have students write at least three specific observations as comments — model the required specificity explicitly: not “Unit Price has some missing values” but **“Unit Price is dtype object, not numeric, and is missing 2 of 42 values (4.8%)”** — the lab page’s own required format. Point out, if it’s present in your data as it is in the synthetic sample: Unit Price being object when it should be numeric is itself a strong hint something’s wrong with the raw values (Exercise 4 reveals it’s “\$”-prefixed strings mixed with plain numbers), *before* students have even looked at the actual data yet — dtype alone is diagnostic.

Common student mistakes to watch for:

- Writing vague observations (“this column looks bad”) instead of the required specific, evidence-cited format — redirect explicitly to the “column X has Y% missing” / “column Z is dtype object but should be float” templates the lab page itself provides.
- Confusing `.isnull().sum()`’s per-column totals with a *row* count — a common early mis-read; have a student state explicitly what the number next to “Unit Price” means (“this many rows are missing a value in this specific column,” not “this many rows are entirely broken”).

Check for understanding: “Which column’s missing-value count worries you *least*, and why?” (A good answer names whichever column has the fewest missing values relative to its total, or one where a missing value is least consequential to later analysis — getting students to triage, not just enumerate, previews Exercise 3’s actual decision-making.)

6.3 Exercise 2 — Standardize Column Names (0:12–0:18, 6 min)

Teaching goal: A single vectorized string operation applied to column *names* themselves — a small, high-payoff cleanup step that makes every subsequent line of code shorter and less error-prone.

Say to the class:

“Notice the column names — mixed case, spaces. That means every reference to them later needs exact capitalization and quoting a space-containing string. One line fixes this permanently.”

Live-code this:

```
# --- Exercise 2 ---
df.columns = df.columns.str.lower().str.replace(' ', '_')
print(df.columns.tolist())
```

Line-by-line explanation:

- `df.columns` — the current column names, as seen in Exercise 1.
- `.str.lower()` — pandas’ `.str` accessor applies a string method (here, Python’s ordinary `.lower()`) to **every column name at once** — say explicitly: this is the same vectorized idea as Module 13 Exercise 5’s `df['revenue'] / df['quantity']`, just operating on the column *labels* rather than the data values.
- `.str.replace(' ', '_')` — chained onto the lowercased result, replacing every space with an underscore — “Order Date” becomes “order date” after `.lower()`, then “order_date” after this step.
- `df.columns = ...` — **reassigns** the DataFrame’s column labels to this new, cleaned list — say explicitly this changes `df` itself, in place; every reference to a column from this point in the script forward must use the new lowercase-underscore names.

Run it. Expected output (verified against the synthetic sample):

```
['order_id', 'order_date', 'product_category', 'region', 'unit_price', 'quantity_sold', 'customer_id']
```

Common student mistakes to watch for:

- Forgetting this changes `df` permanently, and continuing to reference `df['Order ID']` (old capitalization) later in the script — raises `KeyError`, since that column name no longer exists; a good, quick diagnostic if it happens.
- Only calling `.str.lower()` without the `.str.replace()` — leaves spaces in place, which technically still work with bracket notation (`df['order date']`) but prevent the more convenient dot-notation access (`df.order_date`) some pandas code uses; worth a brief mention that this exercise’s two-step cleanup enables that convenience, even if this lab doesn’t require using it.

Check for understanding: “If a column were named 'Customer ID' with multiple spaces between words, would this exact one-liner produce a clean 'customer_id', or something else?” (Something else — 'customer__id', with three underscores, since `.str.replace(' ', '_')` replaces every individual space character one-for-one, not collapsing runs of spaces into one. A good moment to note real column names are often messier than a single clean example suggests, and this simple fix doesn't handle every possible case — good enough for today's dataset, not a universal solution.)

6.4 Exercise 3 — Handle Missing Values (0:18–0:28, 10 min)

Teaching goal: The lab’s first genuine judgment call — drop rows missing essential identifiers, but fill (rather than drop) numeric columns with few missing values — and require students to articulate *why* each choice fits its specific situation.

Say to the class:

“Two different strategies for two different kinds of missingness, and this is a real decision, not a formula. A missing `order_id` means we fundamentally don’t know what transaction this row represents — there’s no reasonable way to guess it, so we drop it. A missing `unit_price`, on a dataset where it’s only missing in a couple of rows, can reasonably be estimated from the other rows’ average — so we fill it instead of losing the whole row.”

Live-code this:

```
# --- Exercise 3 ---
print("Shape before:", df.shape)

# Drop rows missing essential identifiers – no reasonable way to recover
# an unknown order_id or order_date, and keeping a row we can't uniquely
# identify or place in time would corrupt any later grouping/sorting.
df = df.dropna(subset=["order_id", "order_date"])
print("Shape after dropping essential-missing rows:", df.shape)

# Fill unit_price with the column mean – only 2 of 42 rows are missing
# it, and a reasonable price estimate is far less damaging here than
# losing two otherwise-complete, usable rows.
df["unit_price"] = df["unit_price"].astype(str).str.replace("$", "", regex=False)
df["unit_price"] = pd.to_numeric(df["unit_price"], errors="coerce")
mean_price = df["unit_price"].mean()
df["unit_price"] = df["unit_price"].fillna(mean_price)

print(df.isnull().sum())
```

Line-by-line explanation:

- `df.dropna(subset=["order_id", "order_date"])` — `.dropna()` removes rows containing missing values; `subset=[...]` restricts the check to *only* the named columns, rather than dropping a row for a missing value in *any* column (which would be far too aggressive here, given `unit_price` is also sometimes missing but shouldn’t cause a drop). Say explicitly: **without `subset`, `.dropna()` checks every column**, which is almost never what you actually want on a real, multi-column dataset.
- The comment above the drop is the exercise’s real required content — a strong version

names the specific reason essential identifiers can't be reasonably estimated, in contrast to a price.

- `df["unit_price"].astype(str).str.replace("$", "", regex=False)` — this line is doing preparatory cleanup *before* the fill can even happen: since `unit_price` is object dtype (Exercise 1's diagnosis), some values are "\$xx.xx"-style strings. `.astype(str)` ensures every value (including any that might already be numeric) is treated as text for this step; `.str.replace("$", "", regex=False)` strips the dollar sign. `regex=False` is worth a brief explicit note: `$` has a special meaning in regular expressions (end-of-string), and `regex=False` tells pandas to treat it as a literal character to search for instead — omitting this wouldn't necessarily break this specific case, but it's a good habit worth naming when doing literal string replacement.
- `pd.to_numeric(df["unit_price"], errors="coerce")` — converts the now-dollar-sign-free strings to actual numbers; `errors="coerce"` means **any value that still can't be converted becomes NaN** (missing) rather than crashing the whole operation — worth flagging explicitly as the same “handle the unexpected gracefully” philosophy as Module 10's `try/except`, just expressed differently here.
- `df["unit_price"].fillna(mean_price)` — **this has to happen after** the string-to-numeric conversion, not before — say explicitly why: `.fillna()` fills genuinely missing (NaN) values, and before conversion, the “missing” values were sitting alongside dollar-sign strings in an object column where `.mean()` couldn't even be computed correctly in the first place.

Run it. Expected output (verified against the synthetic sample):

Shape before: (42, 7)

Shape after dropping essential-missing rows: (40, 7)

<code>order_id</code>	<code>0</code>
<code>order_date</code>	<code>0</code>
<code>product_category</code>	<code>0</code>
<code>region</code>	<code>0</code>
<code>unit_price</code>	<code>0</code>
<code>quantity_sold</code>	<code>0</code>
<code>customer_id</code>	<code>0</code>

Common student mistakes to watch for:

- Calling `.fillna()` before stripping the "\$" and converting to numeric — `.mean()` on a column still containing text raises an error, or silently produces a nonsensical result depending on pandas' version and the exact mix of values present; a good moment to trace through *why* order matters here, not just accept “do it in this order.”
- Dropping rows for *any* missing value (forgetting `subset=`) — silently loses more data than intended, including the two rows that were only missing `unit_price` and should have been fixable via fill instead. Compare shapes explicitly: an over-aggressive drop produces a smaller `df.shape` than the correct, targeted version.
- Writing a comment that states *what* was done (“I dropped rows with missing `order_id`”) without

the *why* — this is exactly the gap tonight’s reflection question 2 asks students to self-audit; model the difference explicitly now.

Check for understanding: “If `unit_price` had been missing in 30 of 42 rows instead of just 2, would filling with the mean still be a reasonable choice?” (Much less reasonable — a mean computed from only 12 real values, then imposed on 30 rows, would dominate the column with a single estimated number and could seriously distort any later analysis; a good answer recognizes that the “fill vs. drop” decision genuinely depends on *how much* is missing, not just *that* something is missing — directly previewing tonight’s first reflection question.)

6.5 Exercise 4 — Fix Types (0:28–0:38, 10 min)

Teaching goal: `pd.to_datetime(errors="coerce")` and `.astype()` — converting columns to their correct, analysis-ready types, and understanding precisely what happens to a value that can't be converted.

Say to the class:

“Two more type conversions: dates, and quantities. Same `errors='coerce'` philosophy as the price cleanup — a value that can't be converted becomes missing, rather than crashing the whole pipeline.”

Live-code this:

```
# --- Exercise 4 ---
df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")
df["quantity_sold"] = df["quantity_sold"].astype(int)
print(df.dtypes)
print("Unparseable dates (now NaT):", df["order_date"].isnull().sum())
```

Line-by-line explanation:

- `pd.to_datetime(df["order_date"], errors="coerce")` — attempts to parse every value in the column as a real date/time, producing pandas' specialized `datetime64` dtype. `errors="coerce"` means: any value that genuinely cannot be parsed as a date (a malformed string, say) becomes `NaT` (“Not a Time,” the datetime equivalent of `NaN`) rather than raising an error and halting the entire script. Say explicitly: **this is worth checking for after the fact**, since a coerced `NaT` is a new, additional form of missingness this step can *introduce* — one your Exercise 1 diagnosis wouldn't have caught, because it didn't exist until this conversion ran.
- `df["quantity_sold"].astype(int)` — a more direct conversion, since (per Exercise 1's `.info()`) this column was already numeric (`int64`) — this line is included for completeness/safety even though it may be a no-op on data that's already clean in this specific column; worth a brief note that `.astype()`, unlike `pd.to_numeric(..., errors="coerce")`, will *raise an error* on a genuinely unconvertible value rather than silently producing a missing value — a real, meaningful difference in failure behavior worth flagging if a curious student asks why Exercise 3 used `pd.to_numeric` but this line uses `.astype()` directly.

Run it. Expected output (verified against the synthetic sample — one malformed date string was deliberately included):

order_id	object
order_date	datetime64[ns]
product_category	object
region	object

```
unit_price          float64
quantity_sold       int64
customer_id         object
dtype: object
Unparseable dates (now NaT): 1
```

Point out explicitly, since it's a genuinely important consequence of `errors="coerce"`: one row's `order_date` is now `NaT` — a *new* missing value that Exercise 3's `.dropna()` couldn't have caught, since it ran *before* this conversion and the value looked like ordinary (if malformed) text at that point. Ask the room: “should we go back and drop this row too, now that we can see its date is unparseable?” — there's a defensible case either way (drop it, since a missing date is exactly the kind of essential-column gap Exercise 3 already decided to treat as a drop; or keep it, if the row's other information is still independently useful for aggregate statistics that don't depend on date) — the “right” answer matters less than getting students to notice the new problem and reason about it explicitly, exactly the lab's stated grading philosophy.

Common student mistakes to watch for:

- Omitting `errors="coerce"` — `pd.to_datetime()` without it will **raise an exception** on the first unparseable value, halting the whole script; a good live demo of the contrast, since it directly shows what the argument is protecting against.
- Assuming `.astype(int)` would gracefully handle a genuinely non-numeric value the same way `errors="coerce"` does — it would raise `ValueError` instead; worth stating plainly that `.astype()` and `pd.to_numeric(..., errors="coerce")` are not interchangeable safety-wise, even though both can end up producing numeric columns.

Check for understanding: “After this exercise, do we have zero missing values across the whole dataset, or could there still be some?” (Not necessarily zero — the new `NaT` from the coerced date is a fresh missing value nobody has explicitly resolved yet; a good “don't declare victory too early” check that reinforces re-running `.isnull().sum()` after *any* transformation, not just once at the very start.)

6.6 Exercise 5 — Check Duplicates (0:38–0:44, 6 min)

Teaching goal: `.duplicated()` to detect exact duplicate rows, and `.drop_duplicates()` to remove them — a quick, mechanical check worth making habitual on any real dataset.

Say to the class:

“One more common data-quality problem: exact duplicate rows — sometimes from a double-submitted form, a re-run import script, or a copy-paste error somewhere upstream.”

Live-code this:

```
# --- Exercise 5 ---
print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
```

Line-by-line explanation:

- `df.duplicated()` — returns a True/False column, one value per row, True wherever a row is an **exact** repeat of an earlier row (every single column matching) — say explicitly: by default, the *first* occurrence of a repeated row is marked False (kept), and only later repeats are marked True, which matters for understanding what `.drop_duplicates()` actually keeps.
- `.sum()` — Module 13’s familiar boolean-sum trick, here totaling the duplicate flags into a single count.
- `df.drop_duplicates()` — removes the True-flagged rows, keeping one copy of each unique row.

Run it. Expected output (verified against the synthetic sample — two rows were deliberately duplicated during generation):

Duplicate rows: 1

Shape after removing duplicates: (39, 7)

Point out explicitly, if your own dataset’s count differs from what students might expect: the count here (1) may be lower than the number of duplicate rows deliberately introduced upstream, because `.duplicated()` counts *exact, full-row* matches — say plainly, since it’s a genuinely important limitation: **two rows with the same `order_id` but a different `quantity_sold` (say, due to a data-entry correction) would *not* be flagged as duplicates by this check at all**, since `.duplicated()` requires every single column to match. Identifying “the same order recorded twice with conflicting details” is a meaningfully harder, different problem than this exercise covers — worth naming explicitly as a real limitation rather than implying `.duplicated()` catches every kind of duplication.

Common student mistakes to watch for:

- Running `.drop_duplicates()` *before* the type-fixing in Exercise 4 — two rows that are

“the same” except one has "\$72.85" and a since-cleaned version has 72.85 wouldn't be caught as duplicates until after the string-to-numeric conversion made them genuinely identical; worth a brief note on why exercise *order* matters here, not just running each command correctly in isolation.

- Assuming `.duplicated()` flags *all* copies of a repeated row (including the first) — it doesn't, by default; if a student expects the duplicate count to match “how many total rows are part of some duplicate group,” rather than “how many extra copies beyond the first,” walk through the distinction concretely with the actual flagged rows.

Check for understanding: “If a row appeared **three** times, not two, how many would `.duplicated()` flag as True?” (Two — the first occurrence is kept as the canonical copy; the second and third are both flagged as duplicates of it. A good check that the “first kept, rest flagged” rule generalizes beyond the simple two-copy case.)

6.7 Exercise 6 — Describe (0:44–0:53, 9 min)

Teaching goal: Read `.describe()`'s output critically — not just running it, but genuinely inspecting the numbers for values that suggest something worth investigating, and proposing a plausible business explanation.

Say to the class:

“One command, but the real skill is what happens after you run it: does anything in this table look wrong, or at least worth a second look? I want at least two specific numbers flagged, with a guess at what might have caused each one.”

Live-code this:

```
# --- Exercise 6 ---
print(df.describe())

# Suspicious statistics observed:
# 1. [fill in after reading output below]
# 2. ...
```

Line-by-line explanation:

- `df.describe()` — computes summary statistics (count, mean, min, the 25th/50th/75th percentiles, max, and standard deviation) for every **numeric** column at once — say explicitly this automatically skips text columns like `product_category`, since none of these statistics are meaningful on non-numeric data.

Run it. Expected output (verified against the synthetic sample, after all prior exercises' cleanup):

	unit_price	quantity_sold
count	39.000000	39.000000
mean	101.721673	9.051282
min	-25.000000	1.000000
25%	40.850000	6.500000
50%	84.410000	9.000000
75%	116.715000	12.500000
max	999.990000	14.000000

Walk through what a careful read of this table actually reveals, since this is the exercise's real point:

- **min for unit_price is -25.00.** A negative price should immediately look wrong — say explicitly: prices are essentially never legitimately negative in a raw sales record like this (a *refund* might reasonably be represented as a negative value, but that's a specific, different kind of transaction that this dataset doesn't distinguish anywhere). A strong comment pro-

poses a plausible cause: a data-entry sign error, a miscoded return, or a corrupted import — and, importantly, does **not** just delete the row silently without flagging it, since acting on an unexamined guess is itself a risk.

- **max for unit_price is 999.99**, dramatically higher than the 75% value of 116.72 — a strong comment notices this gap specifically (not just “the max is high” in isolation, but *how far* it sits above the rest of the distribution) and proposes an explanation: a genuine bulk/wholesale order, or a data-entry error (an extra digit, a misplaced decimal).
- **The mean (101.72) sitting noticeably above the median (50%, 84.41)** is itself a diagnostic signal worth naming, even as a secondary observation: a mean pulled well above the median suggests the distribution is **right-skewed** — a few large values (like that 999.99 outlier) are pulling the average upward, while most orders cluster lower. This is a genuine preview of Module 15 Exercise 3’s skew discussion, worth flagging as a connection.

Common student mistakes to watch for:

- Running `.describe()` and moving on without genuinely reading the numbers — the lab page’s own grading note is explicit that this exercise is evaluated on “analytical thinking, not just code execution”; if a student’s comment just restates a number without an accompanying “and here’s why that’s worth a second look,” redirect them back to the actual observations above as a model.
- Proposing an explanation with false certainty (“the negative price is definitely a typo”) rather than a *plausible, appropriately hedged* one — a strong analyst’s comment acknowledges genuine uncertainty about root cause while still being specific about the most likely explanations.

Check for understanding: “If you discovered the negative price really was a data-entry sign error, what would you do about it — and how is that decision different from Exercise 3’s drop-vs-fill choices?” (This is a *third* kind of judgment call, distinct from drop or fill: **correct** the value if the true value can be confidently inferred (e.g., flip the sign back to positive) — but only if that inference is genuinely well-supported, not just convenient; otherwise, flagging it for human review, rather than silently “fixing” a value you’re not actually sure about, may be the more honest choice. Getting a student to articulate this as a *third* category — beyond simple drop/fill — shows real analytical maturity.)

6.8 Exercise 7 — Save (0:53–1:01, 8 min)

Teaching goal: Export the cleaned data — and encounter a genuinely important, surprising pandas gotcha: **not every dtype survives a save-and-reload round trip through CSV**, specifically datetime columns.

Say to the class:

“Last step: save the cleaned data, then reload it and verify the types are still correct. I want you to genuinely predict, before we run it, whether you expect every dtype to come back exactly as it was.”

Live-code this:

```
# --- Exercise 7 ---
df.to_csv('data/clean_sales.csv', index=False)

reloaded = pd.read_csv('data/clean_sales.csv')
print(reloaded.info())
```

Run it. Expected output (verified against the synthetic sample):

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 39 entries, 0 to 38
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   order_id              39 non-null    object 
 1   order_date            38 non-null    object 
 2   product_category     39 non-null    object 
 3   region               39 non-null    object 
 4   unit_price           39 non-null    float64
 5   quantity_sold        39 non-null    int64  
 6   customer_id          39 non-null    object
```

This is the exercise’s genuine payoff — read it slowly with the room: `order_date` is now dtype `object`, **not** `datetime64` — even though `df["order_date"]` was correctly converted to a real datetime type back in Exercise 4, and even though it was saved in that state. **Explain why, explicitly:** a CSV file is plain text — it has no way to record “this column is a datetime type” as metadata alongside the values; when a date is written to CSV, it’s written as a *text string* that happens to look like a date (“2024-04-09”), and `pd.read_csv()`, by default, reads every column as whatever type it can most simply infer from the raw text — dates come back as generic strings unless explicitly told otherwise.

Show the fix:

```
reloaded_fixed = pd.read_csv('data/clean_sales.csv', parse_dates=['order_date'])
print(reloaded_fixed.dtypes)
```

Run it — order_date now correctly comes back as datetime64. Say explicitly, as the exercise's real lesson: **.to_csv()/pd.read_csv() is not a perfectly lossless round trip for every dtype** — numeric types generally survive fine, but datetime columns need to be explicitly told to read_csv() via parse_dates=[...] on the way back in, every single time a CSV is reloaded. This is a genuinely common, real source of confusing bugs professionally — a script that worked perfectly when the DataFrame was built fresh can mysteriously fail later when the *same-looking* data is reloaded from a saved CSV, purely because a .dt accessor method (which only works on real datetime columns) is called on what's now just a string column.

Common student mistakes to watch for:

- Assuming the reload's .info() output should exactly mirror the saved DataFrame's .dtypes from before saving — expect some genuine surprise here; that's the intended reaction, not something to smooth over quickly.
- Not noticing the reduced non-null count on order_date after reload (38, not 39) — the NaT value from Exercise 4 was written to the CSV as an empty field, and reads back as a genuine missing value (NaN) either way, dtype issues aside — worth a quick confirmation that this part of the round trip *did* work correctly, to keep the lesson scoped specifically to the dtype issue, not implying the whole save/reload process is unreliable.

Check for understanding: “If this dataset had no date column at all — only text, numbers, and booleans — would this exercise's gotcha have applied at all?” (No, or much less so — numeric and text types generally round-trip through CSV correctly by default; it's specifically datetime (and, relatedly, some other specialized pandas types) that need explicit help on reload. Worth stating plainly that this isn't a reason to distrust CSV broadly, just a specific, memorable exception worth knowing.)

7 Wrap-Up (last ~5 minutes)

Review the reflection questions out loud:

1. *Which cleaning decision felt most ambiguous — drop, fill, or flag? — no wrong answer; a strong response names a specific exercise and articulates what additional information (business context, data-source knowledge) would resolve the ambiguity.*
2. *Would someone unfamiliar with the dataset understand your comments' "why," not just "what"? — this is a genuine self-audit; encourage students to actually re-read their own comments cold, as if seeing them for the first time, before answering.*
3. *The most important thing noticed from .describe() that wasn't expected — likely the negative price, the extreme outlier, or the mean/median gap — push for the specific number and its implication, not just "the data was messier than expected."*

Review the submission checklist together:

- ☐ File is named `clean.py` (or `clean.ipynb`)
- ☐ Contains Exercises 1–7, each clearly separated, with reasoning comments where required
- ☐ Runs top to bottom with no errors
- ☐ Produces `clean_sales.csv` as output
- ☐ Pushed to GitHub inside a `module14/` folder
- ☐ Repo URL submitted to Canvas

Preview Module 15: “Today’s cleaned data becomes next module’s direct input — Module 15 groups it, aggregates it, and turns it into charts answering real business questions, using the exact workflow and standards the capstone itself will grade you on.”

8 Appendix A — Full Answer Key (`clean.py`)

```
# clean.py
# ISM2411 Module 14 Lab – Clean a Messy Sales CSV

import pandas as pd

# --- Exercise 1 ---
df = pd.read_csv('data/messy_sales.csv')
print(df.info())
print(df.isnull().sum())
# Data quality issues observed:
# 1. Unit Price is dtype object, not numeric – some values are "$"-prefixed strings
# 2. Unit Price is missing 2 of 42 values (~4.8%)
# 3. Order ID and Order Date are each missing 1 value – essential identifiers
```

```

# --- Exercise 2 ---
df.columns = df.columns.str.lower().str.replace(' ', '_')
print(df.columns.tolist())

# --- Exercise 3 ---
print("Shape before:", df.shape)
df = df.dropna(subset=["order_id", "order_date"])
print("Shape after dropping essential-missing rows:", df.shape)
df["unit_price"] = df["unit_price"].astype(str).str.replace("$", "", regex=False)
df["unit_price"] = pd.to_numeric(df["unit_price"], errors="coerce")
mean_price = df["unit_price"].mean()
df["unit_price"] = df["unit_price"].fillna(mean_price)
print(df.isnull().sum())

# --- Exercise 4 ---
df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")
df["quantity_sold"] = df["quantity_sold"].astype(int)
print(df.dtypes)
print("Unparseable dates (now NaT):", df["order_date"].isnull().sum())

# --- Exercise 5 ---
print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)

# --- Exercise 6 ---
print(df.describe())
# Suspicious statistics observed:
# 1. min unit_price is -25.00 – prices shouldn't be negative; possible
#    data-entry sign error or miscoded return
# 2. max unit_price (999.99) sits far above the 75th percentile
#    (116.72) – possible bulk order or a data-entry error (extra digit)

# --- Exercise 7 ---
df.to_csv('data/clean_sales.csv', index=False)
reloaded = pd.read_csv('data/clean_sales.csv')
print(reloaded.info())
# order_date comes back as dtype object, not datetime64 – CSV has no
# type metadata; re-read with parse_dates=['order_date'] to restore it.
reloaded_fixed = pd.read_csv('data/clean_sales.csv', parse_dates=['order_date'])
print(reloaded_fixed.dtypes)

```

9 Appendix B — Reproducible Messy Dataset (for instructor testing)

A generator producing a 42-row dataset with deliberate, realistic data-quality issues: mixed-case/spaced column names; five "\$"-prefixed price strings; two missing prices; one missing order_id; one missing order_date; one malformed date string; two duplicated rows (one of which also carries the extreme price outlier); and one negative price. **Use your course's real dataset with students** — this is for your own rehearsal only.

```
import pandas as pd
import numpy as np

rng = np.random.default_rng(7)
regions = ["North", "South", "East", "West"]
categories = ["Electronics", "Home Goods", "Apparel", "Toys"]

rows = []
for i in range(1, 41):
    order_id = f"ORD{1000+i}"
    date = pd.Timestamp("2024-01-01") + pd.Timedelta(days=int(rng.integers(0, 200)))
    rows.append([
        order_id, date.strftime("%Y-%m-%d"),
        rng.choice(categories), rng.choice(regions),
        round(float(rng.uniform(5, 150)), 2),
        int(rng.integers(1, 15)), f"CUST{rng.integers(100,999)}",
    ])

df = pd.DataFrame(rows, columns=[
    "Order ID", "Order Date", "Product Category", "Region",
    "Unit Price", "Quantity Sold", "Customer ID",
])

for i in [2, 5, 9, 14, 20]:
    df.loc[i, "Unit Price"] = f"${df.loc[i, 'Unit Price']}"
for i in [7, 22]:
    df.loc[i, "Unit Price"] = np.nan
df.loc[12, "Order ID"] = np.nan
df.loc[30, "Order Date"] = np.nan
df.loc[18, "Order Date"] = "not_a_date"

df = pd.concat([df, pd.DataFrame([df.loc[3].copy(), df.loc[25].copy()])], ignore_index=True)
df.loc[df.index[-1], "Unit Price"] = 999.99
df.loc[35, "Unit Price"] = -25.00
```

```
df.to_csv("data/messy_sales.csv", index=False)
```

All expected output shown throughout this guide was computed and verified against this exact generator's output.

10 Appendix C — Extra Practice (only if the class finishes early)

Seven required exercises fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — inspect `quantity_sold` for its own quality issues. Even though `.info()` showed it as already `int64` with no missing values, have students check `df["quantity_sold"].describe()` on its own and look specifically for a `min` of 0 or below — a zero-quantity order might indicate a cancelled order that shouldn't be counted in revenue totals, a good extra “does this number make business sense” exercise even on a column that passed the basic type/missingness checks.

Extra — a second duplicate-detection pass, on a subset of columns. Have students run `df.duplicated(subset=["order_id"])` instead of the default full-row check, and compare the count to Exercise 5's result — a good concrete illustration of the “same `order_id`, different other details” limitation flagged in Exercise 5's discussion, since this narrower check would catch cases the full-row version misses.